

EXPERIMENT 10

Static Binary Analysis and Reverse Engineering Using NSA Ghidra

OBJECTIVE

To perform static binary analysis and software reverse engineering of a compiled executable using the NSA Ghidra SRE framework, analyze ELF header structures and compiler metadata, examine disassembly instructions and control flow graphs (CFG), decompile machine code into high-level C pseudocode, extract embedded strings, and verify binary integrity.

TOOLS / REQUIREMENTS

- **Forensic SRE Framework:** NSA Ghidra (Version 12.1.3)
- **Runtime & Build Environment:** OpenJDK 21, GNU Compiler Collection (gcc Version 15.2.0)
- **Analysis Host:** Ubuntu Linux 24.04 LTS (x86_64 architecture on VirtualBox)
- **Target Binary Sample:** Compiled ELF 64-bit Executable (forensic_sample / C program)

EXPERIMENT SCENARIO

Controlled laboratory evidence was used for this experiment.

A controlled static binary analysis scenario was executed to reverse engineer an unknown compiled Linux binary. To establish ground truth without executing unknown code dynamically, a benign C binary (forensic_sample) containing algebraic routines and embedded strings was compiled, hashed, and imported into Ghidra. The investigator analyzed binary headers, mapped function symbols, decompiled subroutines, examined the control flow graph, extracted string artifacts, and audited file integrity.

EVIDENCE / INPUT

- **Target Binary Path:** /home/nandireddy/DIGITAL-FORENSICS/EXP-10/samples/forensic_sample
- **Architecture / Format:** ELF 64-bit LSB PIE (x86-64 / GCC 15.2.0)
- **Target Subroutines Analyzed:** main (Entry point), calculate_score (Arithmetic routine)
- **Embedded String Artifacts:** "Digital Forensics Ghidra Test", "Analysis score: %d\n"
- **Baseline Cryptographic Hash:** SHA-256 (e0e9bc5aa0a671cf262f2dbf8eb3c73fcbdfdf59efcf040c5f4fa81533ad03c7)

PROCEDURE

Step 1 — Initializing NSA Ghidra SRE Framework

Ghidra 12.1.3 was launched on the Linux analysis workstation. The main project manager interface and integrated documentation tools were verified.

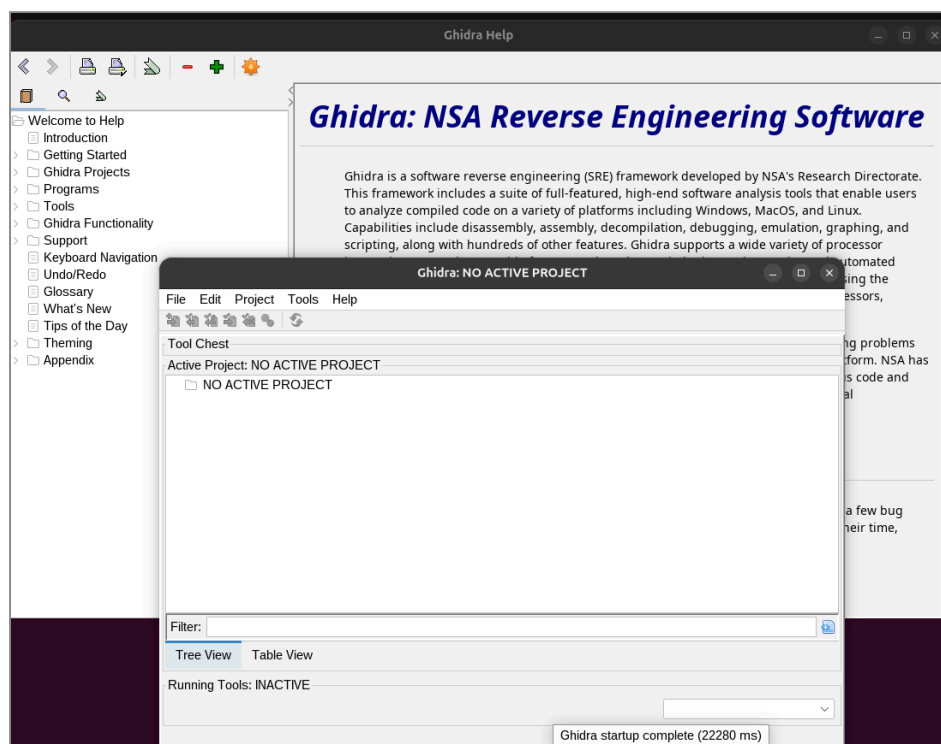


Figure 10.1: Ghidra framework initialization and user interface

Step 2 — Preparing Benign Target Sample, Baseline Compilation, and SHA-256 Hashing

The controlled C test program `forensic_sample.c` was compiled into an x86-64 ELF binary using `gcc`. Baseline execution confirmed expected output and initial SHA-256 hash was recorded.

```
nandiredy@nandiredy-VirtualBox:~/tools/ghidra_12.1.3_PUBLIC$ cd ~/DIGITAL-FORENSICS/EXP-10/samples
cat > forensic_sample.c <<'EOF'
#include <stdio.h>
#include <string.h>

int calculate_score(int value) {
    return value * 2 + 10;
}

int main(void) {
    char message[] = "Digital Forensics Ghidra Test";
    int score = calculate_score(25);

    printf("%s\n", message);
    printf("Analysis score: %d\n", score);

    return 0;
}
EOF
gcc -O0 -o forensic_sample forensic_sample.c

echo "==== FILE INFORMATION ====="
file forensic_sample

echo
echo "==== PROGRAM OUTPUT ====="
./forensic_sample

echo
echo "==== SHA-256 ====="
sha256sum forensic_sample
==== FILE INFORMATION =====
forensic_sample: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=41adc9a5b2156372769
638b64c0210856d340935, for GNU/Linux 3.2.0, not stripped

==== PROGRAM OUTPUT =====
Digital Forensics Ghidra Test
Analysis score: 60

==== SHA-256 =====
ec2d3f1f2a22ba716bac29ae65a31b593870ab53a3313f636f394370dd1443e5 forensic_sample
```

Figure 10.2: Sample source compilation, baseline execution, and initial SHA-256 calculation

Step 3 — Creating Forensic Analysis Project in Ghidra

A dedicated non-shared forensic project Ghidra_Forensics_Project was created to maintain an isolated, organized workspace for evidentiary reverse engineering.

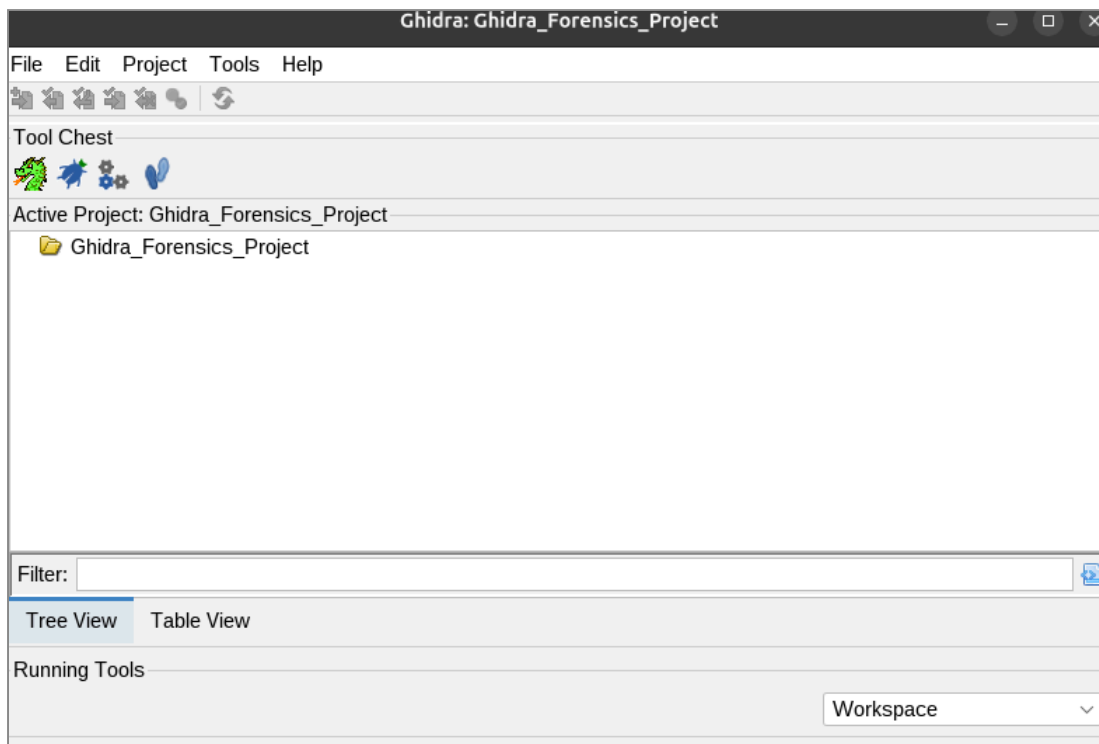


Figure 10.3: Dedicated Ghidra forensics project workspace creation

Step 4 — Ingesting Binary and Analyzing Ingestion Metadata Summary

The forensic_sample binary was imported into Ghidra. The automated format analyzer extracted ELF header metadata, machine type (x86-64), endianness (Little Endian), and compiler identifiers.

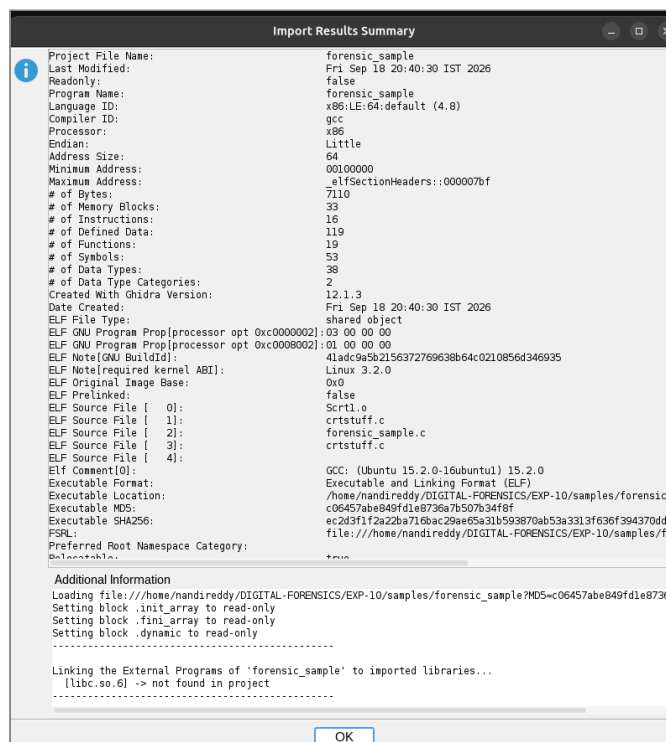


Figure 10.4: Automated ELF binary metadata parsing and Import Results Summary

Step 5 — Symbol Tree Navigation and Function Symbol Enumeration

In the CodeBrowser, the Symbol Tree was navigated to enumerate program functions, locating entry points `_start`, `main`, and the targeted subroutine `calculate_score`.

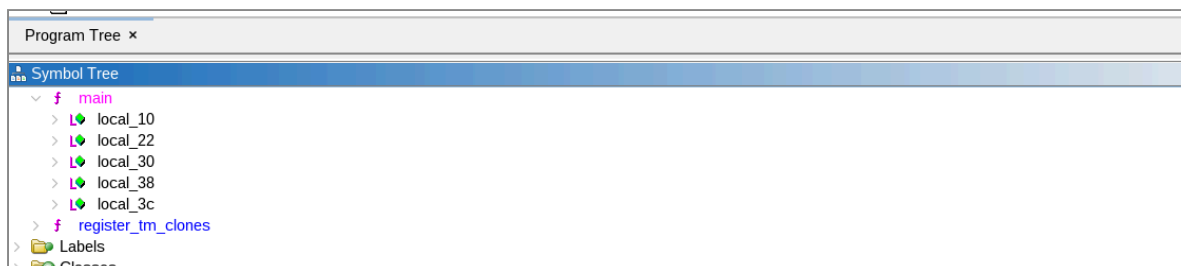


Figure 10.5: Symbol tree navigation and function symbol enumeration

Step 6 — Decompiler Pseudocode Analysis of the main Function

The main function was selected in the Listing window. The Decompiler module automatically reconstructed high-level C pseudocode, displaying variable assignments and function calls.

```
Decompile: main - (forensic_sample)
1
2 undefined8 main(void)
3
4 {
5     uint uVar1;
6     long in_FS_OFFSET;
7     char local_38 [40];
8     long local_10;
9
10    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11    builtin_strncpy(local_38, "Digital Forensics Ghidra Test", 0x1e);
12    uVar1 = calculate_score(0x19);
13    puts(local_38);
14    printf("Analysis score: %d\n", (ulong)uVar1);
15    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
16        /* WARNING: Subroutine does not return */
17        __stack_chk_fail();
18    }
19    return 0;
20 }
21
```

Figure 10.6: Decompiler pseudocode reconstruction for the main function

Step 7 — Static Analysis and Algebraic Decompilation of Subroutine `calculate_score`

The `calculate_score` subroutine was examined. Ghidra accurately decompiled the arithmetic logic (`param_1 * 2 + 10`), proving precise recovery of low-level computational logic.

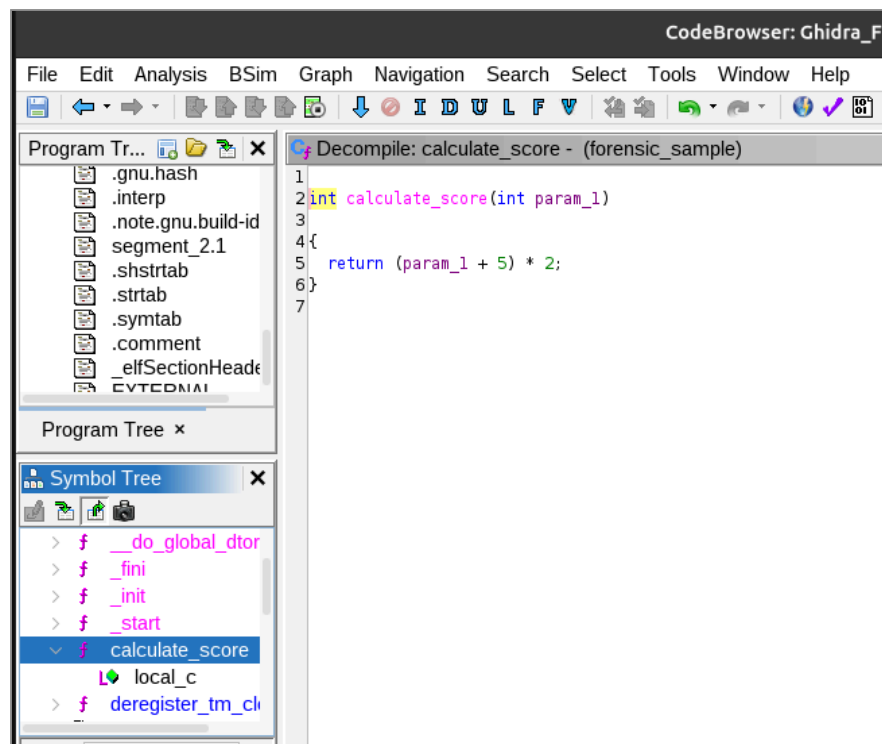


Figure 10.7: Decompiled arithmetic representation of `calculate_score`

Step 8 — Control Flow Graph (CFG) Visual Analysis in Function Graph

The Function Graph view was opened to inspect basic blocks, conditional branching, and control flow architecture of the decompiled functions.

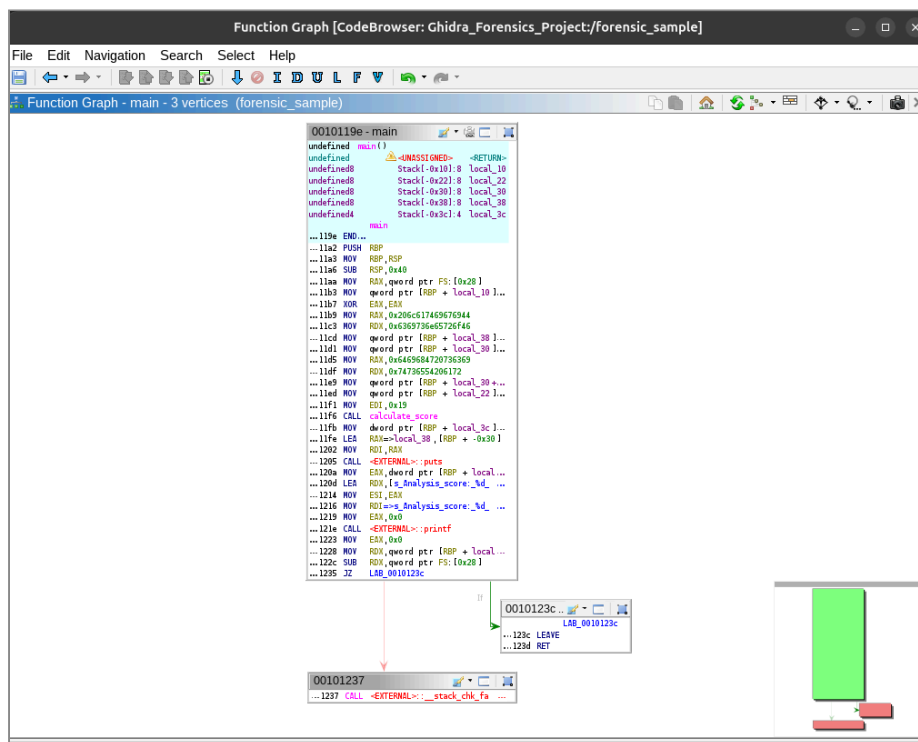


Figure 10.8: Control flow graph (CFG) with basic blocks and branch logic for `main`

Step 9 — Extracting Defined Strings and Static Binary Artifacts

The Defined Strings table was filtered to extract ASCII and UTF-8 strings embedded in the `.rodata` section, identifying forensic strings without executing the binary.

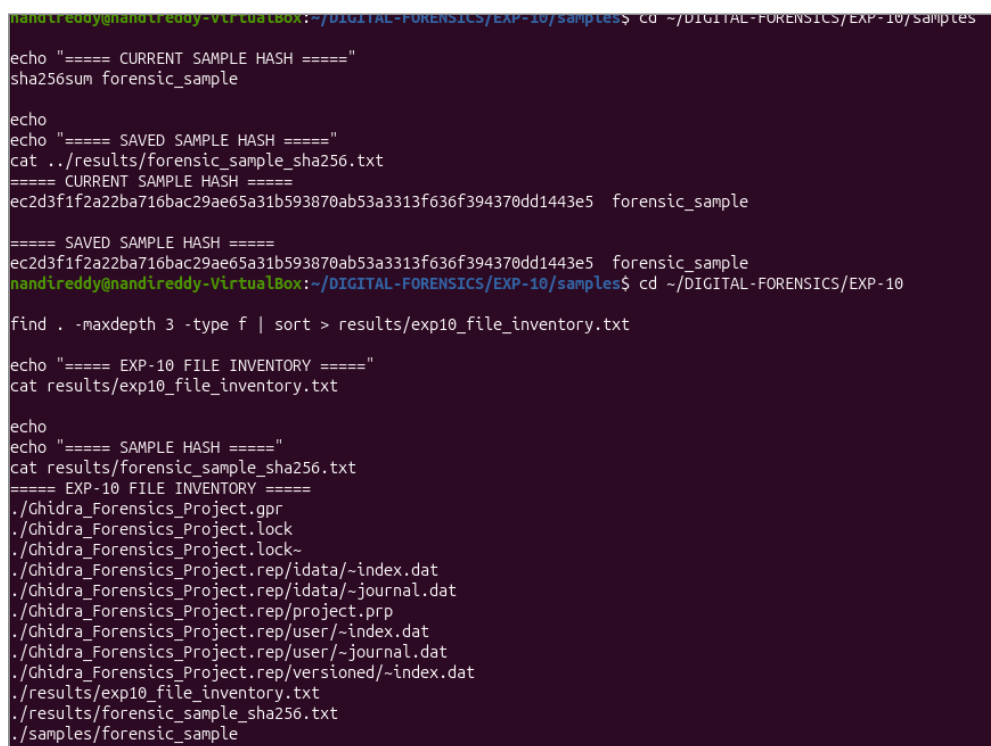


Location	String Value	String Representation	Data Type
00100374	/lib64/ld-linux-x86-64.so.2	"/lib64/ld-linux-x86-64.so.2"	ds
00102004	Analysis score: %d	"Analysis score: %d\n"	ds
00100001	ELF	"ELF"	ds
0010035c	GNU	"GNU"	ds
00102134	GNU	"GNU"	ds
00102164	GNU	"GNU"	ds
00102061	zR	"zR"	ds
.shstrtab::0000010c	.bss	u8".bss"	utf8
.shstrtab::00000111	.comment	u8".comment"	utf8
.shstrtab::00000106	.data	u8".data"	utf8
.shstrtab::000000fd	.dynamic	u8".dynamic"	utf8
.shstrtab::00000048	.dynstr	u8".dynstr"	utf8
.shstrtab::00000040	.dynsym	u8".dynsym"	utf8
.shstrtab::000000ba	.eh_frame	u8".eh_frame"	utf8
.shstrtab::000000ac	.eh_frame_hdr	u8".eh_frame_hdr"	utf8
.shstrtab::0000009e	.fini	u8".fini"	utf8
.shstrtab::000000f1	.fini_array	u8".fini_array"	utf8
.shstrtab::00000036	.gnu.hash	u8".gnu.hash"	utf8
.shstrtab::00000050	.gnu.version	u8".gnu.version"	utf8
.shstrtab::0000005d	.gnu.version_r	u8".gnu.version_r"	utf8
.shstrtab::00000080	.init	u8".init"	utf8
.shstrtab::000000e5	.init_array	u8".init_array"	utf8
.shstrtab::0000002e	.interp	u8".interp"	utf8
.shstrtab::000000d7	.note.ABI-tag	u8".note.ABI-tag"	utf8
.shstrtab::0000001b	.note.gnu.build-id	u8".note.gnu.build-id"	utf8
.shstrtab::000000c4	.note.gnu.property	u8".note.gnu.property"	utf8
.shstrtab::00000086	.plt.got	u8".plt.got"	utf8
.shstrtab::0000008f	.plt.sec	u8".plt.sec"	utf8
.shstrtab::0000006c	.rela.dyn	u8".rela.dyn"	utf8
.shstrtab::00000076	.rela.plt	u8".rela.plt"	utf8

Figure 10.9: Extracted Defined Strings table showing binary string artifacts

Step 10 — Post-Analysis Cryptographic Integrity Verification

The target executable was hashed after analysis using `sha256sum`, confirming the hash matched the pre-analysis baseline and proving static analysis altered zero bytes of evidence.



```
nandireddy@nandireddy-virtualbox: ~/DIGITAL-FORENSICS/EXP-10/samples$ cd ~/DIGITAL-FORENSICS/EXP-10/samples
echo "==== CURRENT SAMPLE HASH ====="
sha256sum forensic_sample

echo
echo "==== SAVED SAMPLE HASH ====="
cat ../results/forensic_sample_sha256.txt
==== CURRENT SAMPLE HASH =====
ec2d3f1f2a22ba716bac29ae65a31b593870ab53a3313f636f394370dd1443e5 forensic_sample

==== SAVED SAMPLE HASH =====
ec2d3f1f2a22ba716bac29ae65a31b593870ab53a3313f636f394370dd1443e5 forensic_sample
nandireddy@nandireddy-VirtualBox:~/DIGITAL-FORENSICS/EXP-10/samples$ cd ~/DIGITAL-FORENSICS/EXP-10

find . -maxdepth 3 -type f | sort > results/exp10_file_inventory.txt

echo "==== EXP-10 FILE INVENTORY ====="
cat results/exp10_file_inventory.txt

echo
echo "==== SAMPLE HASH ====="
cat results/forensic_sample_sha256.txt
==== EXP-10 FILE INVENTORY =====
./Ghidra_Forensics_Project.gpr
./Ghidra_Forensics_Project.lock
./Ghidra_Forensics_Project.lock-
./Ghidra_Forensics_Project.rep/idata/~index.dat
./Ghidra_Forensics_Project.rep/idata/~journal.dat
./Ghidra_Forensics_Project.rep/project.prp
./Ghidra_Forensics_Project.rep/user/~index.dat
./Ghidra_Forensics_Project.rep/user/~journal.dat
./Ghidra_Forensics_Project.rep/versioned/~index.dat
./results/exp10_file_inventory.txt
./results/forensic_sample_sha256.txt
./samples/forensic_sample
```

Figure 10.10: Final SHA-256 integrity verification and file inventory

OBSERVATIONS

1. Ghidra initialized and imported the ELF 64-bit binary into project workspace `Ghidra_Forensics_Project`.

2. The decompiler accurately recovered program logic, variable assignments, and standard I/O function calls (`puts`, `printf`).
3. Subroutine `calculate_score` was reconstructed into clean high-level pseudocode matching the original algorithm.
4. Embedded ASCII string artifacts were extracted from the `.rodata` section without dynamic execution.
5. Pre- and post-analysis SHA-256 hashes matched identically, confirming evidence was preserved unaltered.

FINDINGS

- **High-Level Code Reconstruction:** Ghidra decompiled machine instructions into accurate C pseudocode, recovering control flow, parameters, and algorithms without requiring dynamic execution or debugger attachment.
- **Evidentiary Non-Destructiveness:** Static reverse engineering allowed comprehensive behavioral analysis while ensuring the binary artifact remained completely unmodified throughout the investigation.

CONCLUSION

NSA Ghidra proved to be a highly effective framework for static binary reverse engineering and malware triage. The tool reconstructed high-level pseudocode, mapped control flow graphs, identified embedded string artifacts, and preserved evidentiary integrity.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT

Successfully Completed. Static binary analysis was performed on the compiled ELF executable using NSA Ghidra. Binary headers, disassembly, decompiler pseudocode, control flow graphs, and defined strings were analyzed with verified cryptographic integrity.

REFERENCES

- NSA Ghidra Official Documentation (ghidra-sre.org).
- Eagle, Chris, and Kara Nance. *The Ghidra Book: The Definitive Guide*. No Starch Press.